



Carnegie Mellon University

Master of
Software Engineering

Runtime views (C&C views), pipe-and-filter style

LG Architecture Training Program

Paulo Merson

Agenda

➔ Runtime views

Pipe-and-Filter style

Peer-to-peer style

Interceptor pattern

REST style

Async messaging, Pub-sub

Other messaging patterns

EDA

Software architecture

*The software architecture of a system is the **set of structures** needed to reason about the system, which comprise **software elements, relations among them**, and properties of both.*

[Clements10]

Runtime views (aka C&C views) ₍₁₎

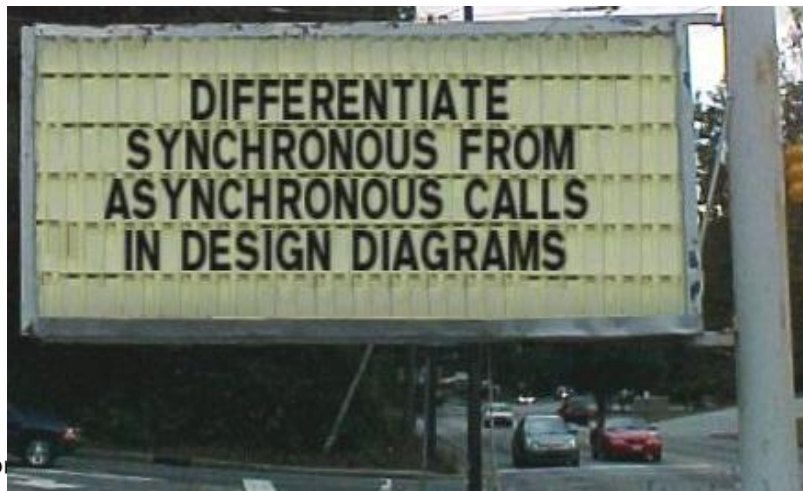
- Show elements with runtime presence, more specifically **components** and their interaction mechanisms, called **connectors**
- Examples of components:
 - REST service, message queue consumer, batch application, process, thread, interceptor
 - Data repository
 - GRPC service, GraphQL service, Kafka consumer, ASP.NET page, ADO.NET dataset
- Examples of connectors:
 - Procedure call, http call, TCP call
 - Pipes, message queues/topics (asynchronous communication)
 - Shared memory
 - Database connectors, file I/O

A message queue can be seen as a component and send/receive as the connector

Runtime views (aka C&C views) ₍₂₎

- **Ports** are points of interaction of a component
 - Components may have ports of different types
 - Components may have multiple instances of a given port type

Diagrams created to explain how the system works are usually runtime views



What do runtime views tell you

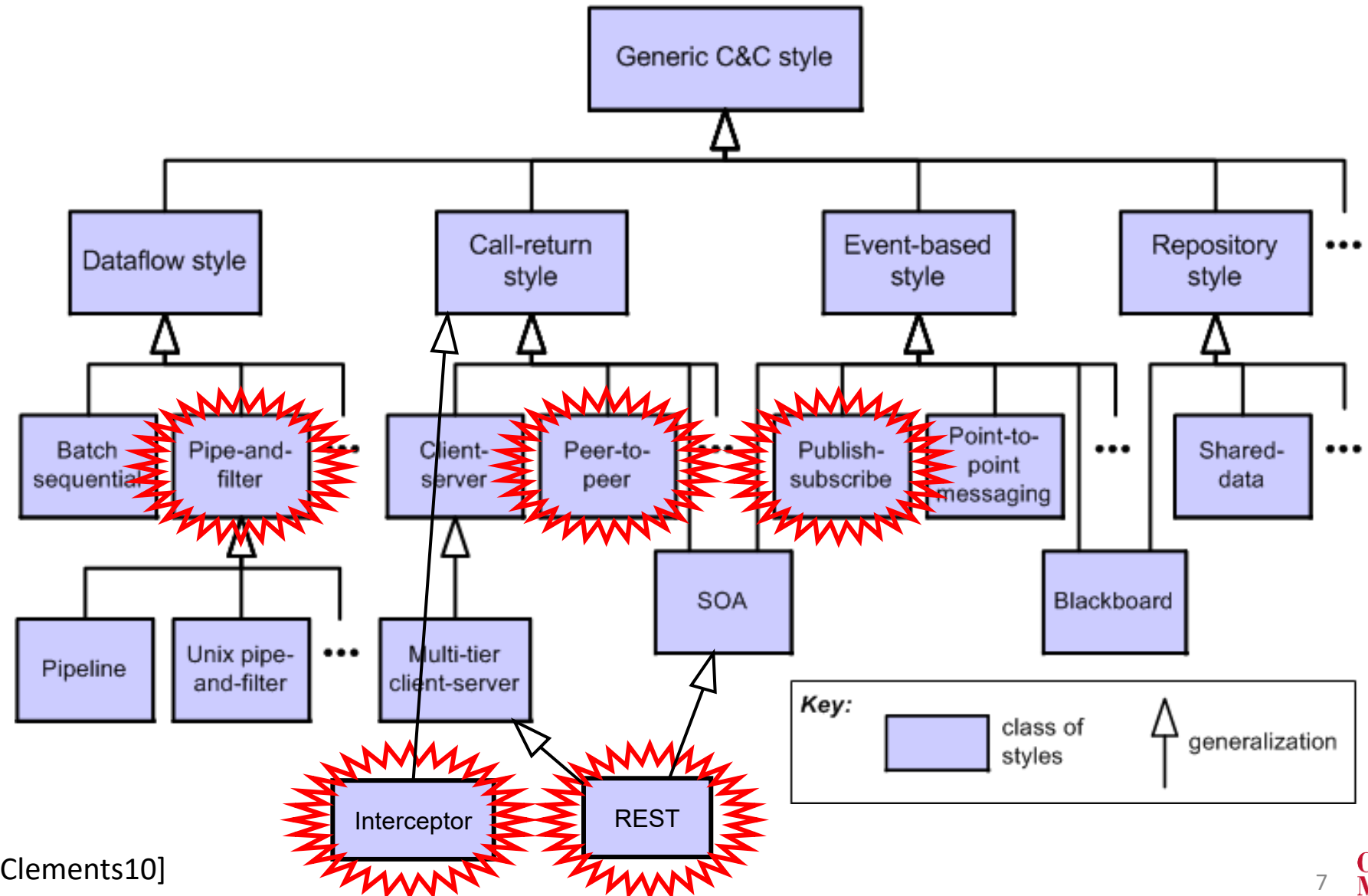
- What are the main components, the type of each one, and how do they interact
- What are the data repositories
- What components are replicated
- What components are used to process a request
 - What infrastructure components participate as intermediaries
- What's the life cycle of components
 - Instantiation, deactivation, reactivation, destruction
- What calls are local or remote



Runtime architecture styles

A given design may use more than one style/pattern

Covered in class



Source: [Clements10]

Agenda

Runtime views

➔ Pipe-and-Filter style

Peer-to-peer style

Interceptor pattern

REST style

Async messaging, Pub-sub

Other messaging patterns

EDA

Pipe-and-filter style

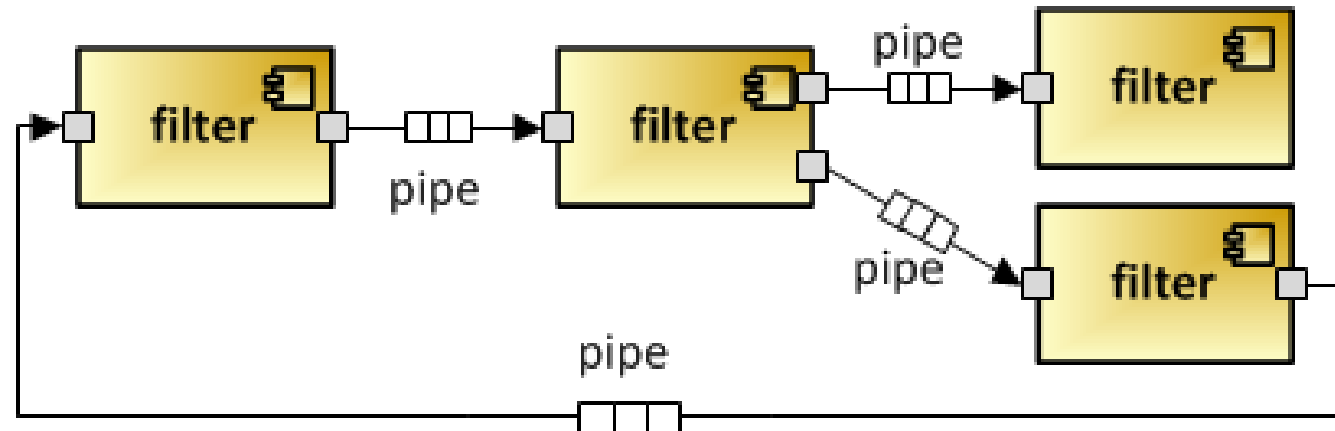
- Component: **filter**
 - Receives data through an input port, transforms the data, and sends the result to an output port
- Connector: **pipe**
 - Unidirectional data conduit with limited capacity
 - Preserves the sequence of data items
- Filters run concurrently; pipes act as buffers between them

In a distributed system the pipes might be message queues in a message broker



Pipe-and-filter topology

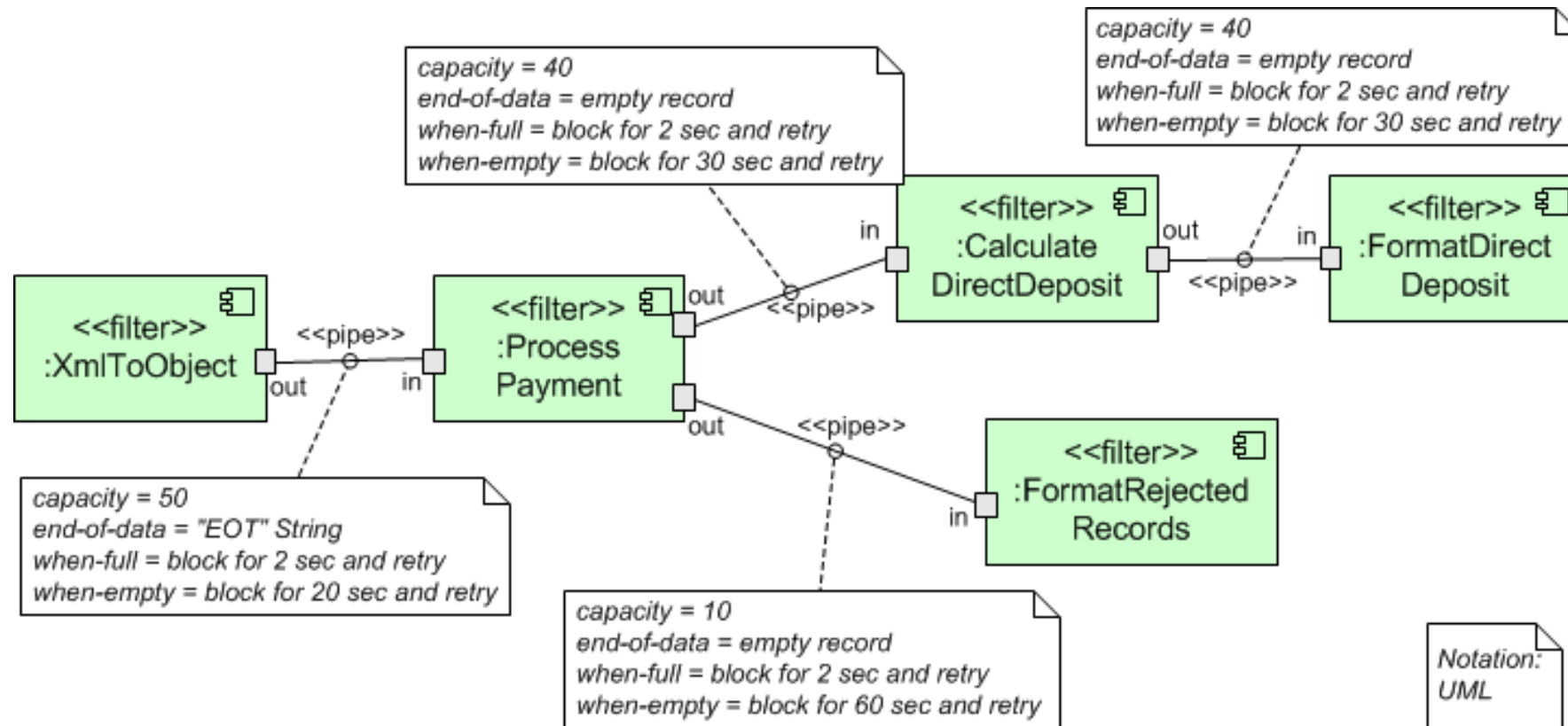
- A pipe connects an output port of a filter to an input port of the next filter, forming a pipeline
- A filter may have more than one input or output ports
- The design may or may not allow
 - Branching and merging
 - Loops



Pipe-and-filter uses and benefits

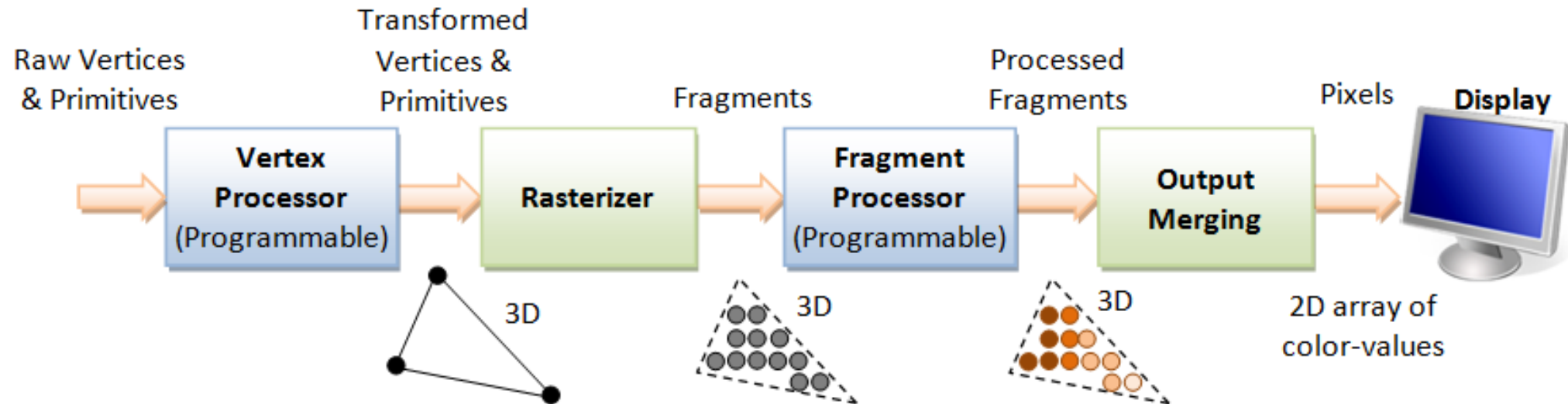
- Commonly used in data transformation systems
- Filters are loosely coupled and can be combined in different topologies
- The pipe-and-filter style promotes
 - Composability
 - Reuse
 - Throughput (due to concurrent processing)

Pipe-and-filter in UML



Source: [Clements10]

Pipe-and-filter example 1



3D Graphics Rendering Pipeline: Output of one stage is fed as input of the next stage. A vertex has attributes such as (x, y, z) position, color (RGB or RGBA), vertex-normal (n_x, n_y, n_z) , and texture. A primitive is made up of one or more vertices. The rasterizer raster-scans each primitive to produce a set of grid-aligned fragments, by interpolating the vertices.

Source: www3.ntu.edu.sg/home/ehchua/programming/opengl/CG_BasicsTheory.html

Where is the notation key?



Pipe-and-filter example 2

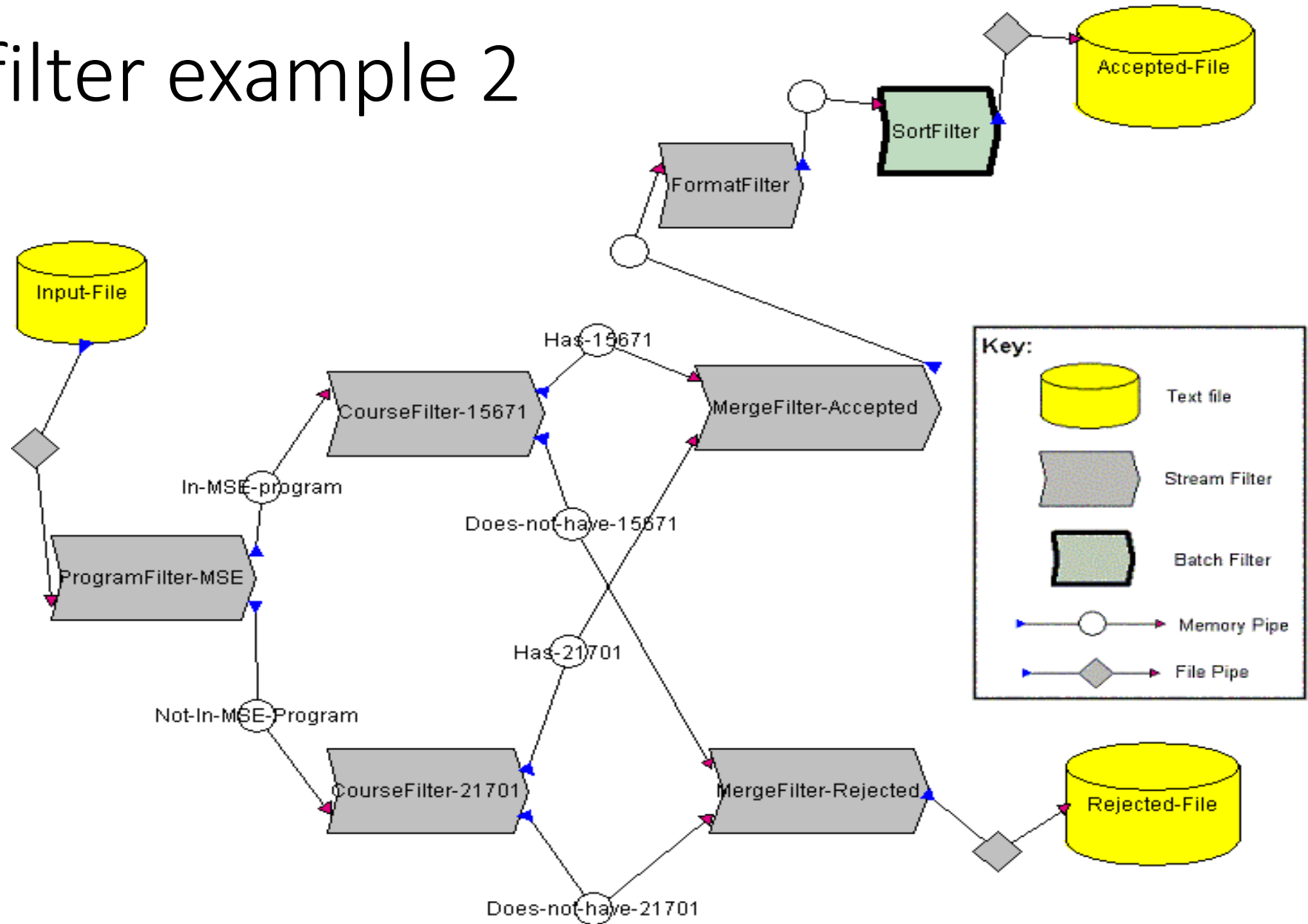
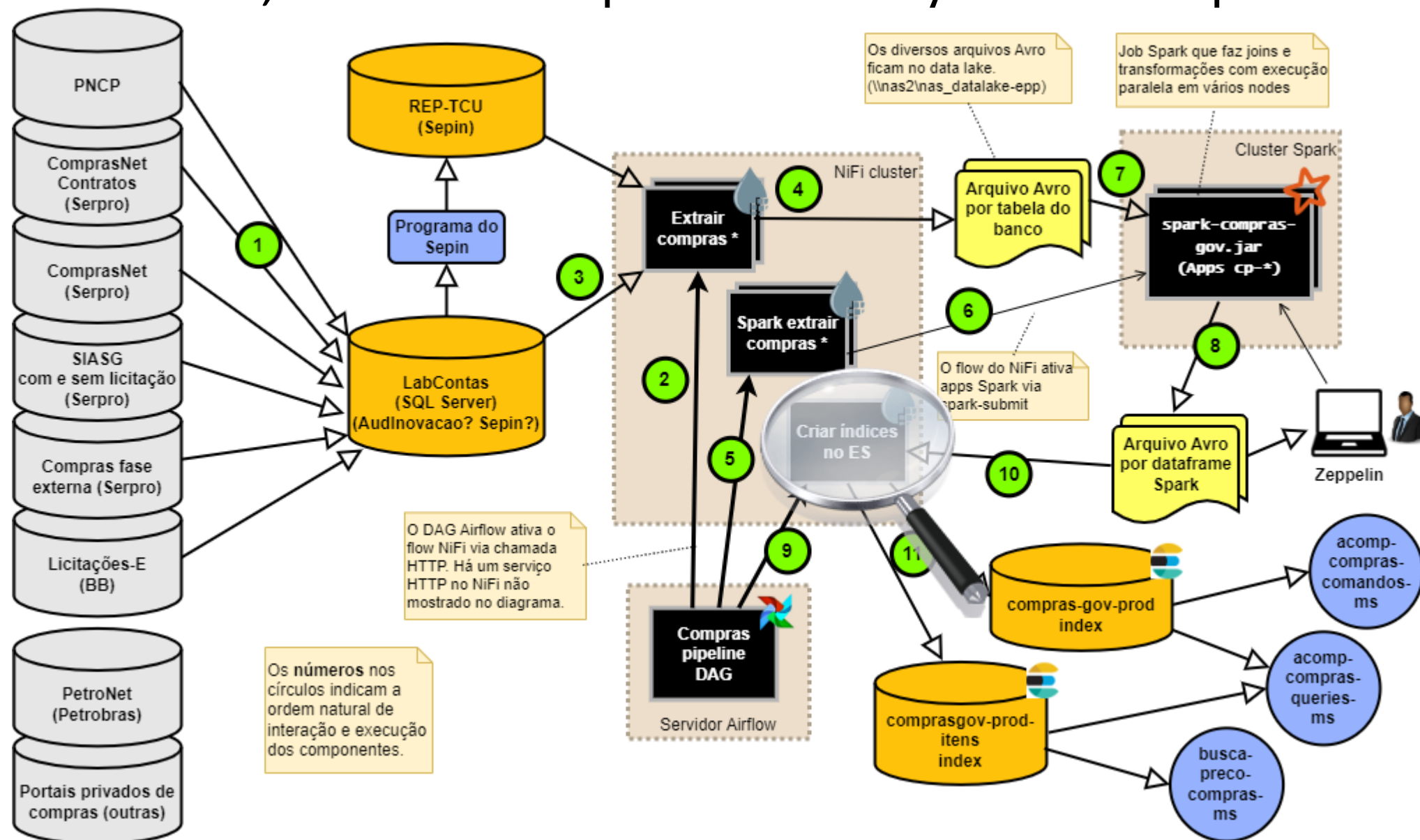


Diagram created
with AcmeStudio

Data-flow, batch sequential style example



Legenda

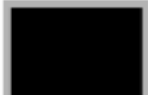
 **serviço REST**
(ou outro serviço request-response)

 **serviço consumidor de tópico**

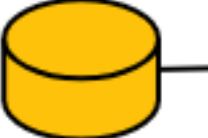
 **serviço REST externo**

 **componente genérico**

 **programa externo**

 **programa batch**

 **banco de dados**

 **LER dados**

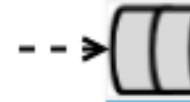
 **ESCREVER dados**

 **arquivo (file system)**

 **artefato de deploy**

 **tópico Kafka**

 **consumir msg do tópico**

 **produzir msg para o tópico**

 **http(s)**

 **conector genérico**

 **ambiente de runtime**

Pipe-and-filter within the data flow example, using NiFi (1)

Filters are called processors and pipes are connections (with a queue)

Carga OpenSearch AWS desenvol

Inicia Dev OpenSearch
GenerateFlowFile 2.6.0
org.apache.nifi - nifi-standard-nar

In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Name **success**
Queued 0 (0 bytes)

GenerateFlowFile
GenerateFlowFile 2.6.0
org.apache.nifi - nifi-standard-nar

In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Name **success**
Queued 0 (0 bytes)

Mudar mime type
UpdateAttribute 2.6.0
org.apache.nifi - nifi-update-attribute-nar

In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Name **success**
Queued 0 (0 bytes)

TO-DO: alterar 'refresh_interval' do index para -1 (desligar) antes da carga e retornar

Extrair atributos provenientes da...
EvaluateJsonPath 2.6.0
org.apache.nifi - nifi-standard-nar

In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

De qual JSON vem o valor '\$.elasticsearch.uri'?

Name **matched**
Queued 0 (0 bytes)

Name **matched**
Queued 0 (0 bytes)

Subdiretório 'all'

Subdiretório compras-comprasnet-contrato.

Lista diretório

Lista diretório

Pipe-and-filter within the data flow example , using NiFi (2)

Data package size is configured via SplitRecord (or SplitText) processors

	Lista diretório ExecuteStreamCommand 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

De onde vem o valor `'${directory}'`?

Name **output stream**
Queued 0 (0 bytes)

	Lista arquivos SplitText 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Name **splits**
Queued 0 (0 bytes)

	Pega nome arquivo avro ExtractText 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Name **matched**
Queued 0 (0 bytes)

	Fetch File FetchFile 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

	Lista diretório ExecuteStreamCommand 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Name **output stream**
Queued 0 (0 bytes)

	Lista arquivos SplitText 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

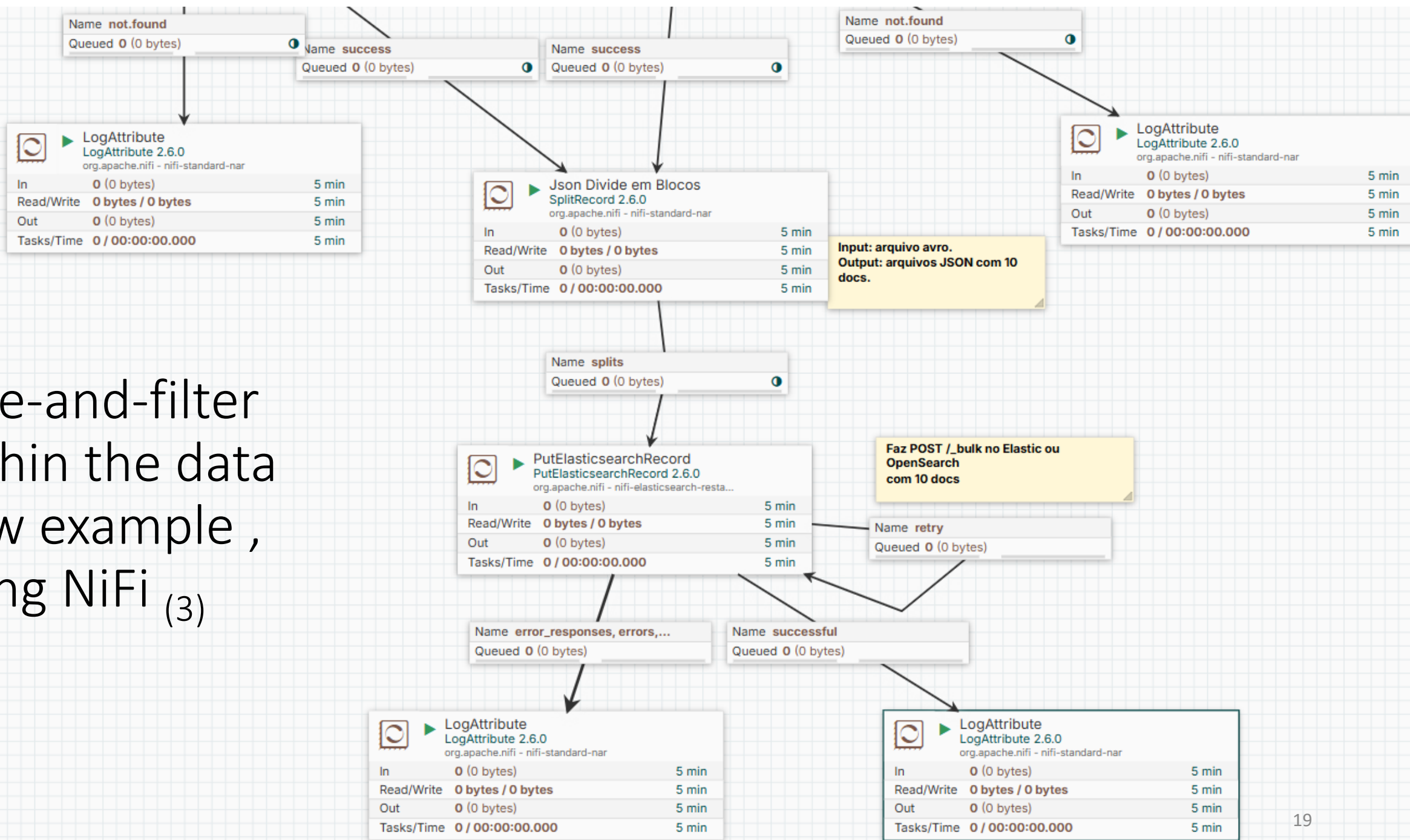
Name **splits**
Queued 0 (0 bytes)

	Pega nome arquivo ExtractText 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Name **matched**
Queued 0 (0 bytes)

	Fetch File FetchFile 2.6.0 org.apache.nifi - nifi-standard-nar	
In	0 (0 bytes)	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 (0 bytes)	5 min
Tasks/Time	0 / 00:00:00.000	5 min

Pipe-and-filter within the data flow example , using NiFi ⁽³⁾





Questions?

Paulo Merson
pmerson@cmu.edu